

1. What is a Controller?

A Controller receives HTTP requests.

Example:

```
@RestController
@RequestMapping("/users")
public class UserController {
}
```

Think of it as:

Browser → Controller → Service

Controller should **not** contain business logic. It only receives requests and returns responses.

2. @RestController

Meaning: This class contains REST APIs.

Without it: Spring ignores the class.

Example:

```
@RestController
public class UserController {
}
```

3. @RequestMapping

Sets the base URL.

Example:

```
@RequestMapping("/users")
```

All methods inside start with /users

4. @GetMapping

Creates GET /users

```
@GetMapping
public List<User> getUsers() {
}
```

5. @PostMapping

Creates POST /users

Example:

```
@PostMapping
public User createUser() {
}
```

6. @PutMapping

Creates PUT /users/5

Example:

```
@PutMapping("/{id}")
public User updateUser(@PathVariable Long id) {
}
```

7. @DeleteMapping

Creates DELETE /users/5

Example:

```
@DeleteMapping("/{id}")
public void deleteUser(@PathVariable Long id) {
}
```

8. Path Variables

Example URL: GET /users/5

Controller:

java

 Copy  Download

```
@GetMapping("/{id}")
public User getUser(@PathVariable Long id) {
}
```

Spring automatically extracts `5` and stores it inside `id`

9. Request Body

Suppose frontend sends:

json

Copy

Download

```
{
  "username": "Shakir",
  "email": "abc@gmail.com"
}
```

Controller:

java

Copy

Download

```
@PostMapping
public User createUser(@RequestBody User user) {
}
```

Spring automatically converts JSON → Java Object



10. Query Parameters

URL: GET /users?page=1&size=10

Controller:

java

Copy

Download

```
@GetMapping
public List<User> getUsers(
    @RequestParam int page,
    @RequestParam int size
) {
}
```

Spring extracts page=1 , size=10

11. ResponseEntity

Instead of:

java

Copy

Download

```
return user;
```

Use:

java

Copy

Download

```
return ResponseEntity.ok(user);
```

Allows control over:

- Status Code
- Headers
- Body

Example:

java

Copy

Download

```
return ResponseEntity.status(201).body(user);
```

12. Request Flow

Suppose browser calls: GET /users

Browser → Spring Boot → Security Filter → UserController → UserService → UserRepository → Hibernate → PostgreSQL → Hibernate → Repository → Service → Controller → JSON Response → Browser

13. Layered Architecture

Controller → Service → Repository → Database

Layer	Responsibility	Example
Controller	Receives requests	@GetMapping
Service	Business logic	userService.getUsers()
Repository	Database access	userRepository.findAll()

14. Why Separate Layers?

Bad (Everything Mixed):

java

Copy

Download

```
@RestController
public class UserController {
    // SQL
    // Business Logic
    // HTTP
}
```

✗ Everything mixed. Hard to maintain.

Good (Separated):

text

Copy

Download

Controller → Service → Repository

✓ Each layer has one responsibility.

15. Example Project — GET /users

Controller:

java

Copy

Download

```
@GetMapping
public List<User> getUsers() {
    return userService.getUsers();
}
```

Service:

java

Copy

Download

```
public List<User> getUsers() {
    return repository.findAll();
}
```

Repository:

java

 Copy  Download

```
public interface UserRepository extends JpaRepository<User, Long> {  
}
```

Hibernate executes `SELECT * FROM users;` automatically.

16. Request Lifecycle (Full Flow)

Client → HTTP Request → Tomcat → Spring DispatcherServlet → Security Filter → Controller → Service → Repository → Hibernate → Database → Hibernate → Repository → Service → Controller → JSON → HTTP Response → Client

17. Common HTTP Status Codes

Code	Meaning
200	Success
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

18. Best Practices

✅ Controllers should be thin.

Good:

java

Copy

Download

```
@GetMapping
public List<User> getUsers() {
    return service.getUsers();
}
```

Bad:

java

Copy

Download

```
@GetMapping
public List<User> getUsers() {
    // SQL
    // Validation
    // Business Logic
}
```

19. Interview Questions

Question	Answer
What is REST?	A style of designing HTTP APIs.
Difference between GET and POST?	GET retrieves. POST creates.
What is JSON?	A lightweight data exchange format.
What is @RestController?	Marks a class as a REST controller that handles HTTP requests and returns data (typically JSON).
Why use Service Layer?	To separate business logic from controllers.
Why use Repository Layer?	To separate database operations from business logic.